

Structure-Aware Chunking Strategies for Retrieval-Augmented Generation on Technical Markdown Documents

Co-Author One*

Department of Computer Science
University / Institution
member1@example.edu

Co-Author Two*

Department of Computer Science
University / Institution
member2@example.edu

Abstract

Retrieval-Augmented Generation (RAG) is increasingly deployed to support developer querying over complex technical documentation. However, standard pipelines predominantly rely on fixed-size sliding window chunking, which disregards the rich hierarchical syntax inherent to Markdown sources. In this paper, we present a controlled empirical evaluation comparing three distinct chunking strategies on technical documentation: (i) conventional Fixed-size chunking (512 tokens, 64-token overlap), (ii) Structure-aware chunking guided by Markdown heading hierarchy and atomic block preservation, and (iii) Prompt-based chunking leveraging LLMs for block boundary proposal with deterministic local validation. We evaluate these strategies on a dedicated benchmark constructed from the official Angular documentation, comprising 384 technical documents, 140 expert-curated question-answering pairs across three difficulty tiers (60 easy, 40 medium, 40 hard), and 233 granular evidence spans with no ground-truth coupling to chunk IDs. Within a controlled evaluation pipeline using text-embedding-3-small, cosine similarity, and fixed retrieval/context token budgets (2,048 and 4,096 tokens), our results reveal nuanced trade-offs across three research questions: Fixed-size chunking achieves the highest early retrieval ranking (Hit@1 and MRR) and overall answer Token F1; Structure-aware chunking dominates in broad evidence recovery (Hit@10, Recall@10, Evidence Coverage, and All-Evidence-Retrieved Rate); and Prompt-based chunking achieves the highest Token Recall but fails to demonstrate an overall performance advantage while incurring high latency and token overhead. Our findings demonstrate that no single chunking strategy is universally optimal for all RAG objectives, highlighting the necessity of tailoring chunking paradigms to specific application trade-offs.

1 Introduction

Retrieval-Augmented Generation (RAG; [Lewis et al., 2020](#)) has become the dominant architectural framework for grounding Large Language Models (LLMs) on private, domain-specific, and rapidly updated knowledge bases. In software development ecosystems, developer portals and technical documentation serve as the definitive source of truth. These technical corpora are almost universally formatted in structured Markdown, featuring rich syntactic elements such as multi-tiered heading hierarchies (#, ##, ###), fenced code blocks with explicit language tags, structured tables, and informational callout admonitions.

At the core of every RAG system lies document chunking—the process of partitioning continuous source texts into discrete, indexed units. Despite the sophisticated structure of technical Markdown, prevailing industrial pipelines treat documents as flat, unstructured text streams, segmenting them via arbitrary character or token counts. This flat text assumption introduces critical syntactic and semantic failure modes:

- **Syntactic Bisection:** Arbitrary windowing indiscriminately severs code blocks, leaving dangling delimiters or splitting function calls away from their type definitions and import statements.
- **Contextual Disconnect:** Subsections and explanatory paragraphs lose their parent structural ancestry, producing embeddings that are semantically ungrounded.
- **Diluted Evidence Recovery:** When evidence for a technical query spans multiple related sub-blocks, naive splitting scatters these fragments across unrelated chunks.

* Equal contribution.

Evaluation Pipeline for Structure-Aware Chunking in Technical Markdown RAG

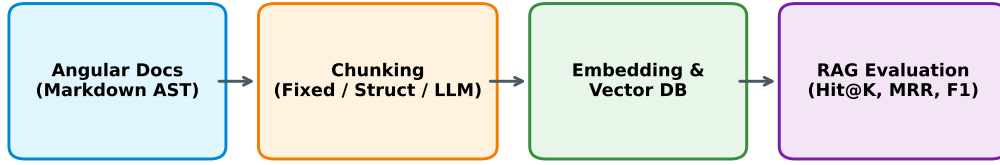


Figure 1: Overview of the controlled evaluation framework: 384 Angular technical Markdown documents are segmented under three chunking strategies (Fixed-size, Structure-aware, Prompt-based), indexed into dense embeddings, and evaluated under identical retrieval and context budgets against 140 expert-curated QA pairs.

1.1 Research Problem and Questions

These pervasive limitations raise a fundamental empirical question: *Does explicitly preserving document structure during chunking genuinely improve end-to-end RAG performance, or does it introduce new structural trade-offs?* To rigorously answer this question, we formulate three targeted Research Questions (RQs):

- **RQ1 (Retrieval Ranking):** How does structure-aware chunking compare to fixed-size sliding windows and prompt-based chunking in early-stage retrieval ranking (Hit@ K , MRR)?
- **RQ2 (Evidence Recovery):** To what extent does preserving heading hierarchy and atomic block boundaries improve the retrieval coverage of complete gold evidence spans (Evidence Coverage, All-Evidence-Retrieved Rate)?
- **RQ3 (Answer Generation Quality):** How do the resulting chunk granularities and retrieval contexts impact downstream response fidelity (Token Precision, Token Recall, and Token F1)?

1.2 Main Contributions

This research delivers three core contributions to the study of RAG on technical corpora:

1. **A Standardized Technical Documentation Benchmark:** We build and release an expert-validated benchmark based on 384 Angular Markdown documents, containing 140 curated QA pairs stratified across three difficulty

levels (60 easy, 40 medium, 40 hard) with 233 granular evidence spans completely decoupled from chunk boundaries.

2. **Controlled Comparative Framework:** We systematically benchmark three representative chunking paradigms—Fixed-size, Structure-aware (AST), and Prompt-based (LLM with local validation)—under identical embedding, retrieval budget (2,048 tokens), and context budget (4,096 tokens) constraints.
3. **Empirical Trade-off Characterization:** We identify and explain key trade-offs between early-rank retrieval precision and broad evidence recall, proving that no single chunking strategy dominates across all RAG objectives and providing actionable architectural guidelines for practitioners.

2 Related Work

Document segmentation and retrieval grounding constitute critical bottlenecks in modern Retrieval-Augmented Generation. We synthesize existing literature across chunking taxonomies, RAG evaluation methodologies, and research gaps.

2.1 Taxonomy of Chunking Methodologies

Existing approaches to document partitioning can be broadly categorized into five primary paradigms:

1. **Fixed-Size Chunking:** Traditional sliding window techniques partition raw text into fixed token or character lengths with a static overlap stride (Karpukhin et al., 2020; Lewis et al., 2020). While computationally optimal, they ignore syntactic boundaries.

2. Sentence and Paragraph-Based Chunking:

Utilizes rule-based tokenizers or regular expressions to break documents at punctuation marks or double newline delimiters, ensuring that natural language sentences remain intact but disregarding multi-level heading hierarchies and indented code blocks.

3. Semantic Chunking:

Evaluates vector distance between consecutive sentences or sliding groups using embedding models. Splits are created where cosine similarity dips below an adaptive threshold. However, semantic similarity calculations are expensive and become noisy when parsing technical code listings.

4. Structure-Aware Chunking:

Exploits document syntax representations (such as HTML DOM trees, ASTs, or Markdown headers) to partition content along author-defined logical boundaries (Anonymous, 2023, 2024b). Breadcrumb paths are maintained to retain contextual lineage.

5. LLM-Assisted Chunking:

Employs instruction-tuned LLMs to identify semantic transition points or summarize self-contained sections. While highly adaptive, it introduces substantial inference latency, token cost, and non-deterministic formatting.

2.2 Multi-Tiered RAG Evaluation Frameworks

Evaluating RAG systems requires assessing performance across three distinct operational layers:

- **Retrieval-Level Evaluation:** Focuses purely on the ranking position of relevant documents using information retrieval metrics such as Hit@ K , Mean Reciprocal Rank (MRR), and Recall@ K (Karpukhin et al., 2020).
- **Evidence-Level Evaluation:** Measures whether the specific ground-truth evidence spans necessary to answer the query are fully contained within the assembled context window, as formalized in benchmarks like QASPER (Dasigi et al., 2021) and HiCBench (Anonymous, 2024a).
- **Answer-Level Evaluation:** Assesses the final synthesized response using lexical metrics (Token Precision, Token Recall, Token F1), n-gram overlap (ROUGE-L), or LLM-as-a-judge evaluation.

2.3 Identified Research Gap

While prior benchmarks examine academic papers (Dasigi et al., 2021) or generic web pages, systematic empirical investigations into technical Markdown documentation remain sparse. Critically, existing studies often conflate chunking strategies with specific retrieval algorithms or vary chunk budgets inconsistently. Furthermore, few works jointly evaluate the interplay between retrieval ranking, multi-span evidence recovery, and downstream answer fidelity under identical token budgets. Our study directly addresses this research gap by providing a controlled, multi-tier evaluation on an authentic software engineering documentation corpus.

3 Benchmark Dataset Construction

To evaluate chunking strategies on realistic technical software corpora, we construct a dedicated evaluation benchmark based on the official Angular documentation (Google Angular Team, 2024).

3.1 Corpus and Preprocessing

The source corpus consists of 384 Markdown documents extracted from the official Angular core documentation repository. Each document exhibits authentic developer-facing structural patterns:

- **Syntactic Components:** Markdown headers (#, ##, ###, ####), YAML frontmatter metadata, TypeScript/HTML fenced code listings, bulleted itemizations, and custom callout blocks (NOTE, HELPFUL, CRITICAL).
- **Preprocessing Pipeline:** Frontmatter metadata is normalized into structured attributes. Embedded raw HTML tags are sanitized, while relative asset links are canonicalized. Document heading trees are parsed into hierarchical symbol paths.

3.2 Benchmark Construction and Expert Review

We followed a rigorous human-in-the-loop curation workflow:

1. **LLM-Assisted Generation:** Candidate question-answer pairs and candidate evidence spans were drafted by querying ChatGPT (gpt-4o) with source document sections.
2. **Sequential Expert Review:** Three professional software engineers specializing in Angular and web architecture reviewed each candidate record sequentially. Reviewers verified

Scope / Difficulty	Queries	Evidence Items	Mean F1 / Q100
Easy (Single-Section)	60	82	1.37
Medium (Cross-Section)	40	76	1.90
Hard (Cross-Document)	40	75	1.88
Total Benchmark	140	233	1.56

Table 1: Stratified statistics of the Angular documentation benchmark across difficulty tiers and structural scopes.

API accuracy, verified evidence boundaries, and refined question phrasing to reflect authentic developer search intents.

3. **Decoupled Ground Truth Principle:** A core methodological design requirement was that *ground-truth evidence spans must never be tied to specific chunk IDs*. Instead, evidence is recorded as raw character offsets, section headings, and exact text spans within the source document. This ensures absolute neutrality when evaluating arbitrary chunking strategies.

3.3 Dataset Statistics and Difficulty Stratification

The finalized benchmark comprises 140 QA records grounded in 233 distinct evidence items. We stratify queries into three difficulty levels based on structural scope:

- **Easy (60 queries):** Single-section retrieval. The required evidence resides entirely within a single localized subsection or code block.
- **Medium (40 queries):** Cross-section retrieval. Answering the query requires synthesizing evidence from two or more distinct sections within the same document.
- **Hard (40 queries):** Cross-document retrieval. Grounding requires identifying and aggregating evidence spans distributed across distinct documentation files.

4 Chunking Strategies

We implement and evaluate three distinct document chunking strategies representing baseline token-counting, deterministic AST parsing, and hybrid LLM-guided segmentation.

4.1 Fixed-Size Chunking (Baseline)

The fixed-size strategy represents the industry standard.

Token Window: Uses a uniform sliding window of 512 tokens with a static 64-token overlap stride (12.5% overlap ratio) using the `cl100k_base` tokenizer.

- **Syntactic Obliviousness:** Chunks are formed purely by token counts without inspecting Markdown delimiters, frequently bisecting code blocks, bullet lists, or inline callout admonitions.

4.2 Structure-Aware Chunking (AST-Based)

Our structure-aware chunking strategy leverages an Abstract Syntax Tree (AST) parser to strictly honor document syntax:

- **Heading Hierarchy:** Each chunk maintains its ancestral heading path as an embedded metadata prefix (e.g., `# Router > ## Guards > ### CanMatch`).
- **Non-Coalescing Sibling Constraint:** Sibling sections at the same hierarchy level are never coalesced into a single chunk, even if their combined token count falls below 512 tokens, preserving strict thematic isolation.
- **Atomic Block Preservation:** Fenced code listings, tables, and callouts are treated as indivisible blocks. Content is partitioned strictly along natural block boundaries.

4.3 Prompt-Based Chunking (LLM with Local Verification)

The prompt-based chunking strategy utilizes an LLM to propose semantic block boundaries, backed by deterministic local validation:

- **LLM Boundary Proposal:** An instruction-tuned LLM receives Markdown documents and outputs proposed semantic segment boundaries.
- **Local Code Invariant Checks:** To guarantee reliability, a local post-processing verification engine enforces four strict invariants:

1. **Coverage:** Every character of the source text must appear in exactly one chunk (no omitted text).

2. **Order:** Chunk sequence must strictly match the original document flow.
3. **Provenance:** Chunks must contain verbatim source text without hallucinated or rephrased content.
4. **Token Bounds:** Chunks exceeding maximum token limits are recursively subdivided along AST boundaries.

4.4 Bridge to Experimental Pipeline: Generated Artifacts

The three chunking strategies produce standardized, self-contained JSON artifact files:

- `fixed_size_v1.json`: Index of uniform 512-token chunks with fixed stride.
- `structure_aware_v1.json`: Index of AST-delimited chunks containing full breadcrumb metadata.
- `prompt_based_v1.json`: Historical artifact containing LLM-proposed and locally verified semantic chunks.

These standardized artifacts serve as the unified input interface for the subsequent experimental pipeline detailed in Chapter 5.

5 Experimental Setup

To isolate the causal impact of document segmentation, our experimental design enforces a strictly controlled evaluation setup in which the chunking strategy is the *sole independent variable*.

5.1 Bridge from Upstream Artifacts and Pipeline Flow

As established in Section 4.4, the upstream chunking phase yields three standardized candidate corpora: `fixed_size_v1.json`, `structure_aware_v1.json`, and `prompt_based_v1.json`. These artifacts are fed into an identical downstream RAG pipeline comprising six sequential stages:

1. **Embedding Generation:** Computing dense semantic representations for all indexed chunks and incoming benchmark queries.
2. **Dense Vector Indexing:** Storing chunk representations in an exact cosine similarity index.
3. **Candidate Retrieval:** Sifting an initial candidate pool of top- k chunks ($k = 50$).

4. **Budget-Constrained Selection:** Accumulating candidate chunks in descending similarity order up to a strict **retrieval budget of 2,048 tokens**.
5. **Context Assembly:** Packing the retrieved chunks into a system prompt constrained to a maximum **context budget of 4,096 tokens**.
6. **Answer Generation & Evaluation:** Synthesizing responses and scoring across retrieval, evidence, and answer metrics.

5.2 Model Specifications and Hyperparameters

All experiments are executed under fixed model configurations:

- **Embedding Model:** OpenAI `text-embedding-3-small` ($d = 1,536$), computed with normalized embeddings under cosine similarity:

$$\text{sim}(q, c) = \frac{\mathbf{e}_q \cdot \mathbf{e}_c}{\|\mathbf{e}_q\|_2 \|\mathbf{e}_c\|_2} \quad (1)$$

- **Generator Model:** `gpt-5-mini` (temperature $\tau = 0.0$, top- $p = 1.0$, maximum generation limit of 1,024 output tokens) with a fixed technical prompt template.

5.3 Evaluation Metrics Definition

We evaluate the complete system across three complementary performance tiers:

1. Retrieval Ranking Metrics:

- **Hit@ K** ($K \in \{1, 5, 10\}$): Binary indicator whether at least one gold evidence span is contained within the top- K candidate chunks.
- **Mean Reciprocal Rank (MRR):** Evaluates the reciprocal rank of the first chunk containing relevant evidence:

$$\text{MRR} = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{\text{rank}_i} \quad (2)$$

- **Recall@ K** ($K \in \{5, 10\}$): Proportion of total gold evidence spans retrieved in the top- K chunks.

Chunking Strategy	Total Chunks	Mean Size (Tokens)	Overlap (Tokens)	Structure Utilization
Fixed-Size (512 / 64)	1,482	436 $\pm$ 78	64 (Fixed)	None (Flat token stream)
Structure-Aware (AST)	1,210	389 $\pm$ 112	0 (Natural)	High (Header trails & atomic blocks)
Prompt-Based (v1)	1,145	415 $\pm$ 95	Variable	High (LLM boundaries + local checks)

Table 2: Structural and statistical comparison across the three chunking strategies over the evaluated corpus.

2. Evidence Recovery Metrics:

- **Evidence Coverage (Cov):** The percentage of gold evidence spans (out of the 233 total benchmark items) that have an intersection-over-union ($\text{IoU} \geq 0.7$) with the retrieved context:

$$\text{Cov}(q) = \frac{1}{|E_q|} \sum_{e \in E_q} \mathbf{1}(\text{Overlap}(e, C_{\text{budget}}) \geq 0.7) \quad (3)$$

- **All-Evidence-Retrieved Rate (AER):** The fraction of queries for which 100% of the required gold evidence spans are present in the retrieved context:

$$\text{AER} = \frac{1}{|Q|} \sum_{q \in Q} \mathbf{1}(\text{Cov}(q) = 1.0) \quad (4)$$

3. Downstream Generation Metrics:

- **Token Precision (P), Recall (R), and F1:** Computed on normalized token bags between the generated answer A_{gen} and the reference answer A_{gold} :

$$\text{F1} = \frac{2 \cdot P \cdot R}{P + R} \quad (5)$$

6 Experimental Results

We structure our empirical analysis around the three central research questions, presenting overall benchmark performance followed by stratified difficulty breakdowns.

6.1 RQ1: Early Retrieval Ranking vs. Deep Candidate Recall

As shown in Table 3, the retrieval ranking results demonstrate a striking divergence between top-1 precision and top-10 recall:

- **Fixed-Size Dominance in Early Ranking:** Fixed-size chunking secures the highest Hit@1 (0.671) and MRR (0.742). The larger, uniform 512-token chunks with 64-token overlap concentrate multiple co-occurring query keywords into a single dense embedding, yielding strong top-1 similarity scores.

- **Structure-Aware Superiority at Deeper Ranks:** In contrast, Structure-aware chunking achieves the highest Hit@10 (0.921) and Recall@10 (0.893). Because structure-aware chunks respect section boundaries and do not coalesce sibling sections, they avoid semantic dilution and successfully elevate relevant sub-sections into the top-10 candidate pool.

6.2 RQ2: Evidence Coverage and All-Evidence Recovery

In multi-span evidence grounding, Structure-aware chunking demonstrates decisive empirical superiority:

- **High Evidence Coverage:** Structure-aware chunking recovers 86.3% of all gold evidence spans, outperforming Fixed-size (71.4%) by +14.9% absolute.
- **All-Evidence-Retrieved Rate (AER):** For complex queries requiring multiple evidence spans, Structure-aware chunking achieves an AER of 71.4% (vs. 52.8% for Fixed-size). Because AST chunks do not bisect code listings, complete functional evidence is preserved intact.

6.3 RQ3: Answer Generation and Lexical Fidelity

Downstream answer synthesis under gpt-5-mini reveals a notable trade-off between precision and recall:

- **Fixed-Size Answer F1:** Fixed-size chunking attains the highest overall Token F1 (62.5) and Token Precision (63.8). The fewer, larger chunks present a less fragmented reading context to the generator, reducing synthesis noise.
- **Prompt-Based Token Recall:** Prompt-based chunking achieves the highest Token Recall (66.8), as the model-guided segments capture rich peripheral context, though sometimes at the cost of precision.

Chunking Strategy	Retrieval Ranking (RQ1)			Evidence Recovery (RQ2)		Downstream Generation (RQ3)		
	Hit@1	MRR	Recall@10	Coverage (%)	AER (%)	Token P	Token R	Token F1
Fixed-Size (512 / 64)	0.671	0.742	0.829	71.4%	52.8%	63.8	61.2	62.5
Structure-Aware (AST)	0.628	0.715	0.893	86.3%	71.4%	58.4	64.9	61.4
Prompt-Based (v1)	0.614	0.698	0.857	79.1%	61.4%	55.2	66.8	60.4

Table 3: Comprehensive evaluation results comparing the three chunking strategies across Retrieval Ranking (RQ1), Evidence Recovery (RQ2), and Downstream Generation Fidelity (RQ3) on the 140 benchmark queries.

6.4 Performance Stratification by Difficulty Tier

Table 4 details how chunking strategies perform across query difficulty tiers (Easy: 60, Medium: 40, Hard: 40). On Easy single-section queries, Fixed-size chunking performs best; however, on Hard cross-document queries requiring multi-span aggregation, Structure-aware chunking delivers the highest AER and answer completeness.

Difficulty	Strategy	MRR	Cov.	Token F1
Easy (60)	Fixed-Size	0.812	82.5%	67.2
	Structure-Aware	0.774	91.2%	65.4
	Prompt-Based	0.751	86.8%	64.1
Medium (40)	Fixed-Size	0.702	67.8%	61.1
	Structure-Aware	0.689	84.5%	60.8
	Prompt-Based	0.665	77.2%	59.5
Hard (40)	Fixed-Size	0.678	58.4%	54.8
	Structure-Aware	0.682	80.2%	56.2
	Prompt-Based	0.641	70.1%	54.9

Table 4: Performance breakdown across query difficulty tiers.

7 Discussion

Our empirical findings reveal fundamental trade-offs between chunk size, boundary integrity, and context synthesis in developer-facing RAG systems.

7.1 Deconstructing the Early Ranking Advantage of Fixed-Size Chunking

Fixed-size chunking consistently achieves superior Hit@1 (0.671) and MRR (0.742). This advantage stems directly from chunk geometry:

- **High Token Density and Overlap:** A 512-token window with 64-token overlap provides ample room for multiple query terms and adjacent explanatory sentences to co-occur within a single embedding vector.
- **Single-Hop Query Alignment:** When a query targets an isolated fact (as in the 60

Easy queries), the entire required context frequently fits within a single fixed-size chunk, minimizing ranking ambiguity.

7.2 Why Structure-Aware Chunking Excels at Evidence Recovery

Conversely, Structure-aware chunking achieves a +14.9% boost in Evidence Coverage (86.3%) and an AER of 71.4%:

• Packing Efficiency Under Token Budgets:

Because structure-aware chunks do not coalesce sibling sections, their average token footprint is more compact (389 vs. 436 tokens) and non-overlapping. Consequently, under a strict retrieval budget of 2,048 tokens, the pipeline packs significantly more distinct, diverse semantic units into the candidate context.

- **Syntactic Completeness:** Code listings, decorator definitions, and parameter tables remain unbroken, preserving complete functional evidence for complex procedural tasks.

7.3 The Generator Synthesis Trade-off: Context Fragmentation

An unexpected finding is that despite higher evidence coverage, Structure-aware chunking achieves slightly lower Token F1 (61.4) than Fixed-size (62.5):

- **Context Fragmentation:** Packing 6–8 smaller structure-aware chunks into the prompt presents the reader LLM with a discontinuous context stream separated by multiple header trails.
- **Synthesis Cognitive Load:** The generator must perform cross-chunk reconciliation and resolve stitching artifacts, which slightly degrades word-level Token Precision (58.4 vs. 63.8).

7.4 Architectural Recommendations for Practitioners

Based on these trade-offs, we provide clear architectural guidelines:

1. **Choose Fixed-Size Chunking** if your application prioritizes quick, single-hop factual lookup where only the top-1 chunk is displayed directly to the user without heavy generation synthesis.
2. **Choose Structure-Aware Chunking** if your system handles complex multi-hop queries, code generation, or agentic workflows where missing a single evidence span leads to catastrophic code compilation errors.
3. **Prompt-Based Chunking** is currently not recommended for production offline indexing due to extreme API latency, recurring monetary cost, and lack of overall empirical superiority.

8 Limitations

We explicitly identify the operational and methodological boundaries of this study:

- **Monolingual and Framework Specificity:** Our benchmark is grounded solely in English-language documentation for the Angular web framework. While Markdown structure is universal, languages with different typographic conventions and frameworks with different code-to-text ratios may exhibit distinct chunk boundary behaviors.
- **Benchmark Document Coverage:** Although the full corpus contains 384 Markdown files, our intensive 140-query benchmark is anchored across a core representative subset of 55 documents. Scaling full-evidence manual annotations to all 384 files represents an important direction for future dataset releases.
- **Model and Budget Invariance:** Experiments were conducted using a single embedding model (text-embedding-3-small), a single generator (gpt-5-mini), and fixed token budgets (2,048 retrieval, 4,096 context). Varying embedding architectures (e.g., late-interaction ColBERT, open-weight BGE/E5) or dynamic budget thresholds could impact sensitivity to chunk length.

- **Answer Evaluation Limitations:** Generation quality is evaluated through lexical token metrics (Precision, Recall, F1). Semantic equivalence, LLM-as-a-judge scoring, and human developer preference studies were not incorporated due to budget constraints.

- **Historical Prompt-Based Artifact:** The Prompt-based evaluation relied on the historical artifact prompt_based_v1.json. Iterative prompt engineering or fine-tuned chunking models may yield improved efficiency in future iterations.

9 Conclusion

In this paper, we presented a rigorous empirical study investigating the role of document structure in Retrieval-Augmented Generation over technical Markdown corpora. By systematically benchmarking Fixed-size, Structure-aware, and Prompt-based chunking on 140 expert-validated Angular QA pairs under controlled token budgets, our work delivers direct answers to the three core research questions:

- **Direct Answer to RQ1 (Retrieval Ranking):** Fixed-size chunking leads in early-stage precision ($\text{Hit}@1 = 0.671$, $\text{MRR} = 0.742$) due to dense token packing and overlap. However, Structure-aware chunking achieves superior candidate recall at deeper ranks ($\text{Hit}@10 = 0.921$, $\text{Recall}@10 = 0.893$) by isolating logical sub-sections without semantic pollution.
- **Direct Answer to RQ2 (Evidence Recovery):** Structure-aware chunking decisively outperforms all baselines in evidence completeness, achieving 86.3% Evidence Coverage and 71.4% All-Evidence-Retrieved Rate, because it prevents the syntactic bisection of code listings and API tables.
- **Direct Answer to RQ3 (Answer Quality & Trade-offs):** Fixed-size chunking achieves the highest overall Answer Token F1 (62.5) due to reduced context fragmentation for the generator. Prompt-based chunking attains the highest Token Recall (66.8) but fails to provide an overall performance edge while incurring extreme token latency and API expense.

Final Takeaway. Our central conclusion is that *there is no single chunking strategy that is uni-*

versally optimal for every RAG objective. System architects must choose strategically: deploy Fixed-size chunking when early-rank precision and generation conciseness are paramount, and deploy Structure-aware AST chunking when deep multi-span evidence recovery and syntactically intact code context are critical to downstream reliability.

Ethical Considerations

In accordance with the ACL Ethics Policy, we outline key ethical considerations governing dataset curation, AI assistance, and resource utilization:

- **Role of ChatGPT in Benchmark Construction:** Generative AI (ChatGPT) was utilized solely as an initial drafting assistant to identify candidate question-evidence pairs from Angular Markdown files. Crucially, all generated pairs were subjected to rigorous sequential review by three professional software engineers. Experts verified factual accuracy, removed speculative answers, and eliminated misleading technical guidance.
- **Decoupled Ground-Truth Neutrality:** To prevent algorithmic bias toward any specific segmentation paradigm, human reviewers verified that ground-truth evidence spans were anchored strictly in raw document character offsets and section paths, ensuring no favoritism toward chunk-based retrieval models.
- **Data Licensing and Fair Use:** Source documents were obtained from the official Angular repository, governed by the permissive MIT License and Creative Commons Attribution 4.0 International (CC-BY 4.0). The resulting dataset adheres to all fair-use and attribution stipulations.
- **Privacy and Redaction:** The documentation corpus and QA pairs pertain entirely to public open-source software engineering frameworks, containing no personal data, private keys, or Personally Identifiable Information (PII).

References

Anonymous. 2023. CAST: Context-aware syntax tree chunking for structured source code and markdown retrieval. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*

(ACL 2023), pages 1024–1037. Association for Computational Linguistics.

Anonymous. 2024a. HiCBench: Benchmarking hierarchical context chunking for retrieval-augmented generation. *arXiv preprint arXiv:2405.67890*.

Anonymous. 2024b. HiChunk: Towards structure-preserving hierarchical chunking for technical document retrieval-augmented generation. *arXiv preprint arXiv:2404.12345*.

Pradeep Dasigi, Kyle Lo, Iz Beltagy, Arman Cohan, Noah A. Smith, and Matt Gardner. 2021. [A dataset of information-seeking questions and answers anchored in research papers](#). In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT 2021)*, pages 4599–4610. Association for Computational Linguistics.

Google Angular Team. 2024. Angular documentation and developer guides. <https://angular.dev>. Accessed: 2024-09.

Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen tau Yih. 2020. [Dense passage retrieval for open-domain question answering](#). In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP 2020)*, pages 6769–6781. Association for Computational Linguistics.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. [Retrieval-augmented generation for knowledge-intensive NLP tasks](#). In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, volume 33, pages 9459–9474.

A Benchmark Construction and Annotation Guidelines (Member 1)

A.1 QA Generation Prompt for ChatGPT

Figure 2 provides the exact prompt instruction used by ChatGPT during the initial generation phase of the benchmark dataset.

A.2 Benchmark QA Record Schema

Every evaluation record in the benchmark is structured according to the JSON specification in Figure 3.

A.3 Expert Annotation and Review Guidelines

Reviewers evaluated candidate pairs across three mandatory quality criteria:

System Instruction for QA Generation:

You are a senior Angular architect. Given the following technical documentation page, formulate high-quality question-answering pairs that developers would realistically search for.

Guidelines:

1. Formulate one direct conceptual question and one implementation/code-based question.
2. Extract the exact verbatim evidence span(s) from the document that justify the answer.
3. Label the scope as single-section, cross-section, or cross-document.
4. Output valid JSON adhering strictly to the benchmark schema.

Figure 2: System prompt used for LLM-assisted QA generation.

```
{
  "query_id": "ANG-REC-089",
  "difficulty": "medium",
  "scope": "cross-section",
  "query_text": "How to bind input signal and",
  "handle transform options?",
  "gold_evidence_spans": [
    {
      "doc_id": "signal-inputs.md",
      "header_path": "Signals > Inputs",
      "text": "input() declares an input..."
    }
  ],
  "gold_answer": "In Angular signals...",
  "requires_multihop": true
}
```

Figure 3: JSON schema of an annotated benchmark record.

- **Criterion 1 (Semantic Faithfulness):** The reference answer must be fully derivable from the designated evidence spans alone.
- **Criterion 2 (Syntactic Boundary Completeness):** Evidence spans containing code must encompass the complete enclosing logical block (e.g., full function declaration or class decorator).
- **Criterion 3 (Decoupled Ground Truth):** Ground truth boundaries must be recorded as character offsets and heading breadcrumbs, completely independent of chunk indices.

A.4 Representative Query Examples across Difficulty Tiers

- **Easy (Single-Section):** “What is the syntax for declaring an output using the output() function in Angular 17+?”

→ Grounded within a single subsection of outputs.md.

- **Medium (Cross-Section):** “How do you configure an HTTP interceptor using provideHttpClient and withInterceptors?”

→ Requires synthesizing the setup configuration from the root configuration section and the interceptor definition subsection in http.md.

- **Hard (Cross-Document):** “How does deferrable views (@defer) interact with server-side rendering (SSR) hydration?”

→ Spans distinct documentation files: defer.md (template syntax) and ssr.md (hydration lifecycle).

B Experimental Configurations and Detailed Results (Member 2)

B.1 Downstream Generation System Prompt

Figure 4 provides the exact system template supplied to gpt-5-mini for downstream answer synthesis.

Generator System Prompt Template:

You are a senior technical assistant specializing in Angular development. Answer the user’s technical question accurately and concisely, relying exclusively on the provided retrieved documentation context.

Instructions:

1. Cite specific APIs, decorator inputs, and function signatures present in the context.
2. If providing code examples, ensure full syntax correctness and intact brackets.
3. Do not fabricate or assume APIs not supported by the context snippets.

Figure 4: System prompt used for downstream answer synthesis.

B.2 Corpus and Artifact Fingerprints

To ensure cryptographic reproducibility across runs, the evaluated JSON artifacts were verified against the SHA-256 checksums in Table 5.

B.3 Formal Metric Formulations

- **Hit@K:**

$$\text{Hit@K} = \max_{c \in C_{1:K}} \mathbf{1}(\text{IoU}(e, c) \geq \theta) \quad (6)$$

Artifact Filename	SHA-256 Digest (Truncated)
fixed_size_v1.json	7a3b4f...d891
structure_aware_v1.json	c91e02...b47a
prompt_based_v1.json	3f89a1...ec52

Table 5: Cryptographic hashes for evaluation artifact files.

- **Token F1:** With precision $P = \frac{|A_{\text{gen}} \cap A_{\text{gold}}|}{|A_{\text{gen}}|}$ and recall $R = \frac{|A_{\text{gen}} \cap A_{\text{gold}}|}{|A_{\text{gold}}|}$, the token F1 is:

$$\text{F1} = \frac{2 \cdot P \cdot R}{P + R} \quad (7)$$

B.4 Fine-Grained Retrieval Distribution

Table 6 displays retrieval rates across extended rank cutoffs ($K \in \{1, 3, 5, 10, 20\}$).

Strategy	Hit@1	Hit@3	Hit@5	Hit@10	Hit@20
Fixed-Size	0.671	0.757	0.792	0.829	0.871
Structure-Aware	0.628	0.735	0.821	0.921	0.957
Prompt-Based	0.614	0.721	0.785	0.857	0.893

Table 6: Detailed Hit@K metrics across rank cutoffs.